Práctica 04.
Aprendiendo Linux.
Gestión de Ficheros



Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera
Copyright: Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional (CC BY-NC-ND 4.0)



Conceptos

puntos de montaje, qué son y para qué sirven; ficheros, estructura de datos nodo-i; directorios, qué son, qué información tienen, cómo se localiza un fichero/directorio mediante una ruta; enlaces, qué son, tipos, en qué se diferencian y por qué son necesarios; órdenes para gestionar ficheros y directorios: copiar, mover, renombrar, borrar, enlazar, crear directorios, encontrar ficheros y directorios.

Contenidos

Contenidos **1**
1. Introducción **2**
1.1. Sistemas de ficheros subsidiarios 2
1.2. Nodos i 4
Mostrar el inodo de un archivo 4
Comprobar el uso de inodos 5
1.3. Directorios y enlaces 5
1.4. Apertura de un fichero 6
2. Enlace de ficheros **8**
3. La orden ln **10**
4. Traslado de ficheros: la orden mv **11**
5. Copia de ficheros: la orden cp **12**
5.1. Copiado no recursivo 13
5.2. Copiado recursivo 14
6. Creación de directorios: la orden mkdir **16**
7. Eliminación de ficheros y directorios: las órdenes rm y rmdir **18**
7.1. Eliminación de ficheros: la orden rm 19
7.2. Eliminación de directorios: la orden rmdir 19
8. Localización de ficheros: la orden find **20**
9. Ejemplos **22**
10. Ejercicios propuestos **25**

Preguntas frecuentes..... 26
inodo o nodo i ¿Qué es un inodo o nodo i?..... 26
Camino o trayectoria ¿Qué es un camino, trayectoria o Path?.....28

1. Introducción

En este módulo se pretende describir con rigor algunas de las **órdenes** de uso más frecuente en UNIX relacionadas con el **sistema de ficheros**. Éstas permiten realizar tareas como **copiar, mover, crear, borrar ficheros y directorios** dentro de la estructura jerárquica o **buscar la ubicación de un fichero** dentro de la jerarquía de ficheros.

En esta sección se describen algunos conceptos de UNIX, como **los enlaces**, claves para entender este módulo.

En Ubuntu 26.04 LTS. Muchas órdenes básicas (ls, cp, mv, ln, rm, rmdir…) proceden de uutils, una reimplementación en Rust. Su comportamiento coincide con el descrito, pero algunos mensajes aparecen en inglés o mezclan los dos idiomas («failed to create symbolic link», «No such file or directory») y ls -l escribe el mes en inglés («Sep»). Los ejemplos de este guion se han obtenido en Ubuntu 24.04 LTS, la versión del aula.

1.1. Sistemas de ficheros subsidiarios

UNIX permite que ciertas **partes (o subárboles) de la jerarquía de ficheros**, llamadas **sistemas de ficheros**, **residan en dispositivos** de almacenamiento o **particiones de disco** aparte. Hay dos razones para hacerlo:

- En primer lugar, el tamaño del disco u otro dispositivo que contenga los directorios y ficheros puede ser insuficiente para almacenar toda la jerarquía, siendo necesario distribuir la jerarquía en varios dispositivos.
- En segundo lugar, los dispositivos de almacenamiento quizá no estén conectados al ordenador de forma permanente. Un ejemplo de sistema de ficheros es una memoria flash (las memorias USB, también llamadas pendrives o llaves USB) que contiene su propia estructura de ficheros.

UNIX gestiona los sistemas de ficheros relacionándolos con **puntos de montaje**. Un punto de montaje **es un directorio en un sistema de ficheros que se corresponde con el directorio raíz de otro sistema de ficheros**.

- El **sistema de ficheros primario** es el que surge del directorio raíz verdadero y se denomina '`/`'.
- Los **sistemas de ficheros secundarios** se enlazan con el sistema primario a través de la orden `mount`, a la que se le da la trayectoria del punto de montaje y la ubicación del sistema de ficheros secundario. Su efecto es hacer que la raíz del sistema de ficheros secundario corresponda al punto de montaje. Una vez que se ha

montado un sistema de ficheros, se puede hacer referencia a cualquier fichero o directorio que contenga utilizando una trayectoria que pase por el punto de montaje. El desmontado (mediante la orden **`umount`**) de un sistema de ficheros invalida todas las trayectorias que pasan por el punto de montaje.

Todo usuario debe ser consciente de los límites del sistema de ficheros por dos razones:

- En primer lugar, existen **restricciones para crear enlaces fuera de un sistema de ficheros** (lo veremos a continuación).
- En segundo lugar, **cuando se desmonta un sistema de ficheros se deja de tener acceso a todos los ficheros** que contiene hasta que se vuelve a montar.

NOTA: En Linux, la orden **mount** muestra todos los montajes activos en el sistema, no solo las unidades físicas o de almacenamiento, sino también sistemas de archivos virtuales para exponer el estado del sistema, puntos de montaje temporales que crean sistemas de archivos en RAM, sistema para instalación de paquetes snap, montajes para depuración y seguridad. El comando **`df -h`** muestra un listado de las particiones y discos físicos que están montados en el sistema y el espacio que ocupan, de forma legible para humanos

En Linux, los sistemas de ficheros se pueden clasificar en reales (o físicos) y virtuales, dependiendo de si residen de forma persistente en un dispositivo de almacenamiento o si son creados y gestionados por el kernel en la memoria RAM.

Sistemas de Ficheros Reales (Físicos)

Estos sistemas de ficheros se encuentran en los discos duros, SSDs, memorias USB, etc. Su objetivo es almacenar datos de forma permanente. Los más comunes que encontrarás en Linux son:

- `ext4`: Es el sistema de ficheros por defecto de Ubuntu y la mayoría de las distribuciones Linux. Es una versión mejorada de ext3, muy robusta y con un buen rendimiento.
- `XFS`: Un sistema de ficheros con alta escalabilidad y rendimiento, muy popular para servidores que manejan grandes volúmenes de datos.
- `Btrfs`: Un sistema de ficheros moderno con funciones avanzadas como la creación de instantáneas (snapshots), sumas de verificación para la integridad de los datos y compresión.
- `NTFS`: El sistema de ficheros nativo de Windows. Ubuntu lo monta para permitir el acceso a particiones de Windows.
- `FAT32`: Utilizado comúnmente en memorias USB y tarjetas SD, aunque tiene limitaciones como el tamaño máximo de archivo (4 GiB). Las memorias USB de gran capacidad suelen usar exFAT, que no tiene esa limitación.

Sistemas de Ficheros Virtuales

Estos sistemas no están en el disco. Son una abstracción del kernel para acceder a información del sistema o a dispositivos. Se montan en ciertos directorios del sistema de archivos, pero solo existen en la memoria mientras el sistema está encendido. Su contenido se borra cuando el equipo se apaga o reinicia.

- `procfs`: Montado en el directorio `/proc`. Proporciona información sobre los procesos en ejecución y la configuración del kernel.
- `sysfs`: Montado en el directorio `/sys`. Expone información sobre el hardware y el kernel de forma estructurada, permitiendo la interacción con los dispositivos.
- `tmpfs`: Utilizado para directorios como `/run` y `/dev/shm`. Almacena archivos temporales y datos del sistema en la memoria RAM para un acceso muy rápido.
- `devtmpfs`: Montado en el directorio `/dev`. Es un sistema de ficheros virtual que el kernel usa para crear los "nodos de dispositivo" que representan a los dispositivos de hardware.
- `cgroup2: Montado en el directorio /sys/fs/cgroup. Utilizado` para la gestión de grupos de control (cgroups), que regulan el uso de recursos de procesos

1.2. Nodos i

En UNIX todo **fichero** tiene asociado una pequeña zona de información en el disco (una estructura de datos) denominada **nodo i** o **inodo**. Cada archivo **se identifica por un número de inodo**. Este número es único dentro de todo el sistema de archivos.

Cuando se **crea un nuevo archivo a este se le asigna un nombre, y además, un número de nodo i**, para que el sistema operativo pueda identificar tu archivo.

Un nodo i contiene toda la información importante para gestionar un fichero, entre otros:

- la **identidad del propietario** del fichero y del grupo (su uid y gid),
- los **permisos** del fichero,
- el **tamaño,**
- las fechas de último acceso (atime), de última modificación del contenido (mtime) y de último cambio del propio nodo i (ctime); ext4 guarda además la de creación,
- las **zonas del disco** donde se ubica el fichero (punteros a los bloques de datos del archivo en el disco),
- la cantidad o **números de enlaces duros** al fichero

Mostrar el inodo de un archivo

Para saber el número de inodo de un archivo, puedes ejecutar la orden **`ls`** con la opción **`-i`** o también **`--inode`**. En el caso de un directorio se usará **`ls -di`**

```
alumno@so-lab01:~$ ls -i
alumno@so-lab01:~$ ls -di
```

La ejecución de este comando listará los archivos contenidos dentro del directorio, junto con el número de nodo i de cada uno de ellos.

```
alumno@so-lab01:~/prueba$ ls -li
total 8148
33753572 -rw-rw-r-- 1 alumno alumno       0 oct  2 21:41 fichero1.txt
33753573 -rw-rw-r-- 1 alumno alumno       0 oct  2 21:41 fichero2.txt
33753574 -rw-rw-r-- 1 alumno alumno       0 oct  2 21:41 fichero3.txt
33753570 drwxrwxr-x 3 alumno alumno    4096 oct  2 21:41 proyecto
33753575 -rw-rw-r-- 1 alumno alumno 8337920 oct  2 21:41 proyecto.tar.gz
```

Comprobar el uso de inodos

Cada sistema de ficheros tiene su conjunto de nodos i para guardar la información asociada a los ficheros del sistema. **Cada sistema de ficheros tiene un número limitado de identificadores disponibles** (números de nodo i), que impone el número máximo de archivos que se pueden crear que no depende del espacio en disco duro. Los nodos i se numeran desde 1; el número 0 no se usa (véase el apartado 1.4).

Para conocer la cantidad de números disponibles, cuántos has usado y el porcentaje que se ha ocupado:

```
alumno@so-lab01:~$ df -i
```

El comando **`df`** muestra información sobre el espacio utilizado y disponible de los sistemas de archivos montados, sin opciones muestra la información en bloques. La opción **`-i`** le indica que en lugar de mostrar la información del espacio usado, muestre información relacionada con los inodos.

Esto quiere decir que nuestra computadora se puede quedar sin espacio de dos formas:

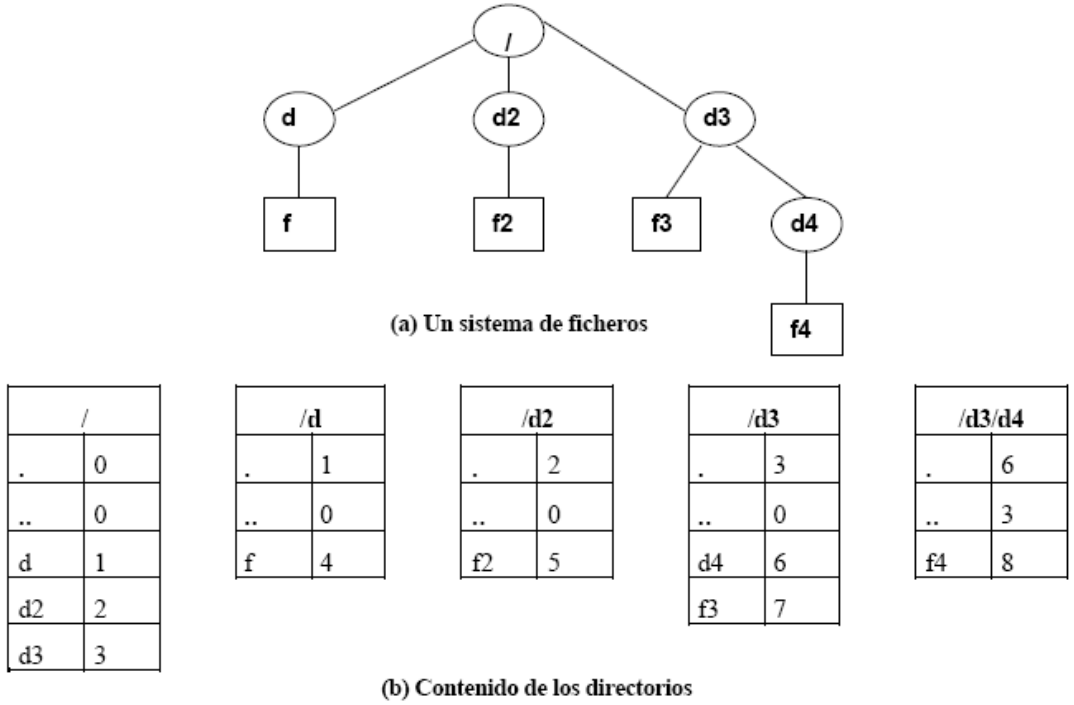
- Ocupamos toda la capacidad del disco duro
- Se agotan los nodos i disponibles en el sistema de ficheros

```
alumno@so-lab01:~$ df -i
S.ficheros     Nodos-i NUsados NLibres NUso% Montado en
tmpfs           502367    1123  501244    1% /run
/dev/sda2      1638400  231472 1406928   15% /
tmpfs           502367       1  502366    1% /dev/shm
tmpfs           502367       5  502362    1% /run/lock
/dev/sda1            0       0       0     - /boot/efi
tmpfs           100473     134  100339    1% /run/user/1000
```

1.3. Directorios y enlaces

En UNIX un **directorio** es un **tipo especial de fichero** que contiene una serie de **entradas** llamadas **enlaces a ficheros**. Un **enlace es un par formado por un nombre**, que nombra al enlace en el directorio, y **un número de nodo i**, que indica con qué fichero está asociado el enlace (nos dice la estructura de datos que contiene información sobre el fichero).

Por ejemplo, la figura 1.b) muestra el contenido de los directorios del sistema de ficheros representado en la figura 1.a).



(a) Un sistema de ficheros

/	
.	0
..	0
d	1
d2	2
d3	3

/d	
.	1
..	0
f	4

/d2	
.	2
..	0
f2	5

/d3	
.	3
..	0
d4	6
f3	7

/d3/d4	
.	6
..	3
f4	8

(b) Contenido de los directorios

Figura 1. (a) Un sistema de ficheros y (b) el contenido de sus directorios

1.4. Apertura de un fichero

1. Los procesos solicitan el uso de los ficheros mediante una llamada al sistema. A esta solicitud se le llama **apertura del fichero**. La llamada precisa como parámetro una trayectoria o ruta del fichero que se quiere abrir.
2. Como respuesta a la solicitud el sistema operativo **localiza el nodo i del fichero en el disco** y, si el proceso tiene permisos para abrir el fichero, **carga su nodo i en la memoria principal**.
3. A partir de entonces, cuando el proceso solicite leer y/o escribir del fichero el sistema operativo obtiene la información necesaria para realizar la lectura o escritura del nodo i almacenado en memoria principal, en lugar de hacerlo de disco, ya que el tiempo de acceso a memoria principal es inferior al tiempo de acceso a disco.

Veamos ahora **cómo localiza el sistema operativo el nodo i de un fichero a partir de la trayectoria proporcionada por un proceso**.

Por ejemplo, supongamos que se emplea la trayectoria `/d2/f2` para el sistema de ficheros de la figura.

1. Como la trayectoria comienza por `/` el sistema operativo inicia su búsqueda en el **directorio raíz**, el cual tiene asociado un nodo i fijo que depende del sistema de archivos y su implementación particular. A estas rutas *(paths)* que comienzan en la raíz del sistema de archivos de una unidad se les suele denominar trayectorias o

rutas absolutas. En los sistemas operativos tipo Unix y Linux, como aquellos que usan los sistemas de archivos ext2, ext3 o ext4, el inodo 0 generalmente no se usa, y el directorio raíz (/) tiene el inodo 2. El inodo 1 suele estar reservado para los bloques defectuosos, mientras que el inodo 2 identifica al directorio raíz y se considera que todos los archivos y directorios parten de él. Sin embargo, para facilitar la explicación, en el ejemplo se usará el nodo i 0 para la raíz.

2. Se consulta el nodo i número 0 para saber a partir de qué dirección de disco se almacena `/`, tras obtener la dirección se lee el contenido de `/` (obsérvese en la figura 1.b), y se busca la entrada de nombre `d2`.
3. Se accede ahora al nodo i de `d2` (el 2) para localizar a partir de qué dirección de disco se almacena su contenido.
4. Una vez descubierto se lee `d2` y se busca la entrada de nombre `f2`. Ésta está **asociada al nodo i 5**.
5. Se consultan en el nodo i 5 los permisos del fichero, y si el usuario que ejecuta el proceso tiene permitido el acceso al fichero se guarda el nodo i en memoria principal.

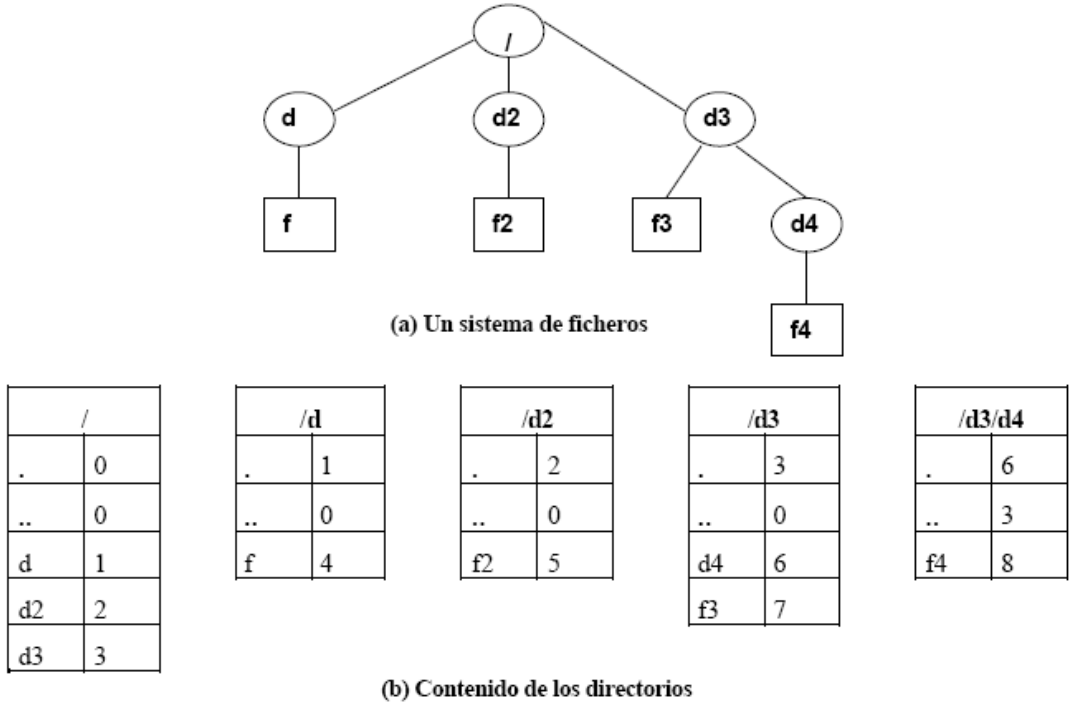


Figura 1. (a) Un sistema de ficheros y (b) el contenido de sus directorios

Con las trayectorias o **rutas relativas** se trabaja de forma análoga, sólo que la búsqueda comienza en el **directorio de trabajo** del proceso, en lugar de comenzar en el directorio raíz. El nodo i asociado al directorio de trabajo de un proceso se almacena en su descriptor o bloque de control de proceso BCP.

En la figura 1 puede observar que todo directorio posee al menos dos entradas: `.` y `..`, que hacen referencia al **propio directorio** y **al padre de éste respectivamente**. Estas entradas se añaden automáticamente cuando se crea un directorio. Su utilidad reside en permitir **construir trayectorias relativas al directorio de trabajo y al directorio padre** de éste.

Por ejemplo, si el directorio de trabajo es **`/d3/d4`** la trayectoria **`../f3`** hace referencia a **`/d3/f3`**. Piense que si el directorio de trabajo está situado a cierta profundidad en el sistema de ficheros y queremos utilizar una entrada de un "directorio hermano" al directorio de trabajo el uso de una trayectoria que incluya el **`..`** será mucho más corta que la trayectoria absoluta.

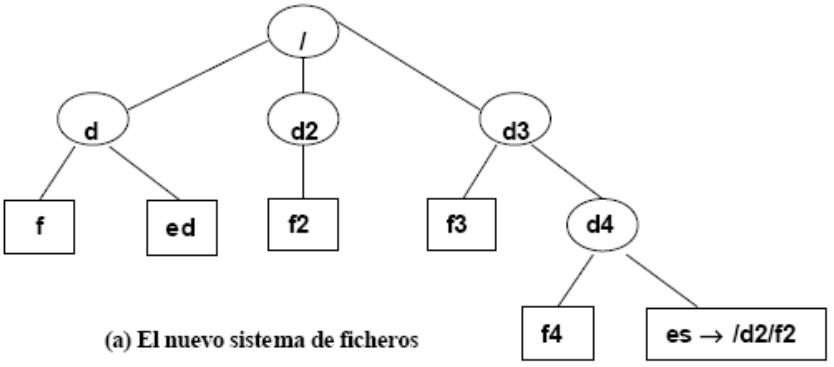
2. Enlace de ficheros

UNIX permite que exista más de un enlace a un fichero o directorio, lo que posibilita el acceso a un mismo fichero a través de distintas trayectorias. Existen dos tipos de enlaces: los enlaces tradicionales o **enlaces duros** (*hard* o físicos) y los **enlaces simbólicos** (*soft* o suaves).

La figura 2.a) representa el mismo sistema de ficheros que el de la figura 1.a), salvo que se han añadido dos enlaces más: **`ed`** y **`es`**.

- **`ed`** es un **enlace duro** al fichero asociado al enlace **`f2`** del directorio **`/d2`**, es decir, al fichero cuyos datos se almacenan en el nodo i 5. Como puede observar tanto la entrada **`ed`** del directorio **`/d`** como la **`f2`** del directorio **`/d2`** hacen referencia al mismo nodo i: el 5.
- Si se tiene en cuenta lo que hace UNIX en el proceso de apertura de un fichero (comentado en la sección anterior) se deduce que la apertura de las trayectorias **`/d/ed`** y **`/d2/f2`** producen el mismo resultado: solicitar el fichero asociado al nodo i 5. En terminología UNIX este fichero posee dos enlaces duros.

Todos los enlaces del sistema de ficheros de la figura 2.a), salvo **`es`**, son duros. Observe que, por ejemplo, el directorio **`/d3`** posee 3 enlaces duros: el enlace **`d3`** de **`/`**, el enlace **`.`** de **`/d3`** y el enlace **`..`** de **`/d3/d4`**.



(a) El nuevo sistema de ficheros

/	
.	0
..	0
d	1
d2	2
d3	3

/d	
.	1
..	0
f	4
ed	5

/d2	
.	2
..	0
f2	5

/d3	
.	3
..	0
d4	6
f3	7

/d3/d4	
.	6
..	3
f4	8
es	9

(b) Contenido de los directorios

Figura 2. (a) El nuevo sistema de ficheros y (b) el contenido actualizado de sus directorios

El nodo i de un fichero almacena el número de enlaces duros que posee el fichero.

Los **enlaces simbólicos**, como los duros, sirven para aportar trayectorias adicionales a un fichero, pero se implantan de manera distinta, por lo que exhiben propiedades diferentes. Un enlace simbólico hace referencia a un pequeño fichero cuyo contenido es una trayectoria del sistema de ficheros.

Por ejemplo, en la figura 2.a) **`es`** hace referencia a un fichero que se almacena en el nodo i 9, de contenido **`/d2/f2`**. Cuando se solicita la apertura de una trayectoria como **`/d3/d4/es`** UNIX procede de forma normal, sin embargo, cuando localiza su nodo i asociado (el 9) se comprueba (leyendo un bit del nodo i) si el fichero se utiliza como enlace simbólico. Si es así UNIX, lee su contenido para obtener la trayectoria enlazada, en el ejemplo **`/d2/f2`**, y localizar su nodo i, es decir, el 5. El resultado final es acceder al nodo i 5, y por tanto, abrir el mismo fichero que se hubiera obtenido empleando la trayectoria /d2/f2. Sin embargo, observe que aunque el proceso de apertura de **`/d2/f2`** y **`/d3/d4/es`** produce el mismo resultado, la segunda trayectoria implica una apertura más lenta, pues implica más accesos a disco por parte del sistema operativo.

Otra diferencia entre enlaces duros y simbólicos viene dada por la posibilidad de realizar enlaces a sistemas subsidiarios. No es posible crear un enlace duro a un fichero de un sistema subsidiario. Esto se debe a que **cada sistema de ficheros numera su conjunto de nodos i del uno o dos en adelante**. Si creamos un enlace duro al nodo i 7 nos referimos al nodo i 7 de nuestro sistema de ficheros. Sin embargo, podemos crear un enlace simbólico a un fichero de un sistema subsidiario usando una trayectoria de enlace que pase por un punto de montaje.

3. La orden ln

La orden **ln** (de *link*) crea un enlace con uno o más ficheros existentes. Tenemos dos formatos de uso:

```
$ ln [-s] [opciones] fich1_destino fich2_enlace
$ ln [-s] [opciones] destino1 destino2 ... dir
```

En la primera forma se crea un enlace entre **fich2** y el fichero enlazado a **fich1**. Si fich2 ya existe, **ln** termina con un error («El archivo ya existe»), salvo que se use la opción -f, que elimina el enlace antiguo antes de crear el nuevo.

En la segunda forma **ln** crea un enlace para cada fichero en el directorio **dir**, asignando al enlace el mismo nombre que tiene el fichero. Como en la primera forma, si ya existe un enlace con ese nombre, **ln** no lo sustituye salvo con la opción `-f`. **Los enlaces duros normalmente solo pueden enlazar archivos, no directorios ni carpetas**.

Cuando **ln** crea un enlace duro, el nombre del enlace destino tiene el mismo conjunto de permisos y el mismo propietario y grupo que el enlace del fichero fuente.

No se permite crear un enlace entre un directorio de un sistema de ficheros y un fichero de otro sistema, a menos que el enlace sea simbólico. Por omisión **ln** crea enlaces duros y no reemplaza ningún enlace existente. Se puede cambiar este comportamiento con las siguientes opciones de la línea de órdenes:

- **-s** Crea enlaces simbólicos en lugar de enlaces duros.
- **-f** Elimina los enlaces de destino que ya existan.
- **-i** Solicita confirmación antes de reemplazar un enlace existente.

Crear un enlace duro

```
alumno@so-lab01:~$ ls -l sync.sh
-rw-r----- 1 alumno alumno 5 oct  2 21:41 sync.sh
alumno@so-lab01:~$ ln sync.sh synchro
```

Ahora comparemos los dos archivos

```
alumno@so-lab01:~$ ls -il sync.sh synchro
33753576 -rw-r----- 2 alumno alumno 5 oct  2 21:41 synchro
33753576 -rw-r----- 2 alumno alumno 5 oct  2 21:41 sync.sh
```

Lo interesante de los enlaces duros es que no hay diferencia entre el archivo original y el enlace: son solo dos nombres conectados al mismo inodo.

Crear un enlace simbólico

```
alumno@so-lab01:~$ ln -s /home/alumno/sync.sh filesync
```

Aquí 'filesync' es un enlace simbólico a 'sync.sh'. Piense en ello como un atajo. Editar 'filesync' es como editar directamente el archivo original, y eso es lo que realmente sucede: se modifica el fichero original. Si eliminamos o movemos el archivo original, el enlace se romperá y nuestro archivo 'filesync' ya no estará disponible.

Si se ejecuta **`ls -l`** se muestra que el archivo resultante es un enlace simbólico. Esto se indica con la letra **`l`** en la primera posición de la `ls -l` y también por la flecha al final de la lista, que indica el archivo al que se refiere el nombre.

```
alumno@so-lab01:~$ ls -l filesync
lrwxrwxrwx 1 alumno alumno 20 oct  2 21:42 filesync -> /home/alumno/sync.sh
```

El contenido de un enlace simbólico es solo el nombre del archivo de destino. Puede ver que los permisos en el enlace simbólico están completamente abiertos. Esos permisos no se utilizan: al acceder a través del enlace se aplican los del fichero de destino.

Al comparar el enlace simbólico y el archivo original, notará una clara diferencia entre ellos.

```
alumno@so-lab01:~$ ls -il sync.sh filesync
33753613 lrwxrwxrwx 1 alumno alumno 20 oct  2 21:42 filesync ->
/home/alumno/sync.sh
33753576 -rw-r----- 2 alumno alumno  5 oct  2 21:41 sync.sh
```

Tienen un número de inodo diferente. El archivo enlace contiene el camino del fichero enlazado, su tamaño es la cantidad de bytes del nombre del fichero al que se hace referencia.

4. Traslado de ficheros: la orden mv

La orden **`mv`** *(move)* pasa uno o más ficheros de un directorio a otro. Si los dos directorios están en el mismo sistema de ficheros, lo único que hace **`mv`** es cambiar sus enlaces, sin transferir datos. Si están en sistemas de ficheros distintos, **`mv`** copia los datos de los ficheros del primer sistema en el segundo, crea los enlaces necesarios en el segundo sistema de ficheros, y luego elimina los enlaces en el primer sistema. El traslado se puede considerar como un cambio de nombre de los ficheros. Cuando se traslada un fichero permanecen intactos su modo de protección y su pertenencia. También se conservan al trasladar entre sistemas de ficheros distintos: mv copia los datos y mantiene los permisos y las fechas; el propietario sólo puede conservarse si quien ejecuta mv tiene privilegios para asignarlo. Las formas de utilizar mv en la línea de órdenes son:

```
$ mv [opciones] fich1 fich2
$ mv [opciones] fichero ... dir
```

En la primera forma se crea un enlace entre `fich2` y el fichero enlazado a `fich1`, y se elimina el enlace original de `fich1`. Si `fich2` ya estaba enlazado a un fichero, se destruye el enlace anterior al crearse el nuevo (esto no es lo mismo que destruir el fichero, ya que se conservan sus demás enlaces).

En la segunda forma, cada fichero en la lista *`fichero ...`* se cambia a `dir`. El traslado conserva los nombres de los ficheros que se han movido.

Por omisión, **`mv`** solicita confirmación cuando reemplaza un enlace con un fichero que carece de permiso de escritura. Se puede modificar este comportamiento con las siguientes opciones de la línea de órdenes:

- **`-i`** Solicita confirmación al reemplazar cualquier enlace, sin importar sus permisos.
- **`-f`** No solicita confirmación al reemplazar un enlace que carece de permiso de escritura.
- **`-n`** No sobrescribe ningún fichero existente.

5. Copia de ficheros: la orden cp

La orden **`cp`** *(copy)* copia uno o más ficheros. A diferencia de `ln` y `mv`, `cp` sí copia los datos y no sólo reacomoda las entradas en el directorio. Los formatos de `cp` de la línea de órdenes son:

```
$ cp [opciones] fichentrada fichsalida
$ cp [opciones] nomentrada ... dirsalida
```

donde `fichentrada` es un fichero de entrada, `fichsalida` es un fichero de salida, cada `nomentrada` es un fichero o directorio de entrada y `dirsalida` es un directorio de salida. Los elementos de entrada se denominan "fuentes"; los elementos de salida se conocen como "destinos".

El primer formato se aplica cuando el destino es un fichero. En este caso sólo puede haber una fuente, un fichero. El fichero de entrada se copia en el fichero de salida.

El segundo formato se aplica cuando el destino es un directorio.

Las copias pueden ser recursivas o no recursivas, aunque la versión de **`cp`** en System V sólo permite copias no recursivas. La opción **`-r`** especifica la recursividad de la copia, la cual sólo afecta a la copia de directorios.

5.1. Copiado no recursivo

```
$ cp [opciones] fichentrada fichsalida
$ cp [opciones] nomentrada ... dirsalida
```

Para una copia no recursiva es necesario que exista el directorio `dirsalida` y que cada `nomentrada` nombre un fichero, y no un directorio. Cada uno de los ficheros de entrada se copia en el directorio de salida y se le asigna el mismo nombre base que tenía originalmente. Por ejemplo, suponga que **`/usr/homer`** y **`burt`** son directorios y que **`/usr/homer`** contiene los ficheros **`lisa`** y **`marge`**. Entonces, la orden

```
alumno@so-lab01:~$ cp /usr/homer/* burt
```

copia **`/usr/homer/lisa`** en un fichero de nombre **`burt/lisa`** y **`/usr/homer/marge`** en un fichero llamado **`burt/marge`**.

Si el fichero de destino no existe, se crea un fichero con ese nombre. Si el fichero de destino ya existe, cp no crea un fichero nuevo: sobrescribe su contenido, y el destino conserva su nodo i. Enseguida se copian los datos de cada fichero fuente en su fichero de destino correspondiente. Después de la copia, los ficheros fuente y destino son independientes, o sea, las modificaciones de un fichero no afectan al otro.

Si el fichero de destino existía, conserva su propietario y sus permisos; si no, se crea con los permisos del fichero fuente reducidos por la umask, y su propietario es el usuario que ejecuta cp. Si el fichero de destino no tiene permiso de escritura, cp no pregunta: informa del error («Permiso denegado») y no copia, salvo con la opción -f, que elimina el destino y crea un fichero nuevo.

Múltiples formas de tratar la sobreescritura mientras se copian los archivos

Probablemente no siempre querrás que tus archivos de destino existentes se sobrescriban y eso es totalmente lógico.

Para evitar la sobreescritura de archivos existentes, puedes utilizar la opción **`-n`**. De esta manera, **`cp`** no sobrescribirá los archivos existentes. En Ubuntu 24.04, cp añade un aviso en inglés: el comportamiento de -n no es igual en todos los sistemas y se recomienda la forma equivalente --update=none.

```
alumno@so-lab01:~$ cp -n archivo_origen directorio_objetivo
cp: warning: behavior of -n is non-portable and may change in future; use
--update=none instead
```

Pero tal vez quieras sobrescribir algunos archivos. Puedes utilizar la opción interactiva **`-i`** y te preguntará si quieres sobrescribir los archivos existentes.

```
alumno@so-lab01:~$ cp -i archivo_origen directorio_objetivo
cp: ¿sobreescribir 'directorio_objetivo/archivo_origen'? (s/n) s
```

Puedes responder s para sobrescribir el archivo existente o `n` para no sobrescribirlo.

También hay una opción para hacer copias de seguridad automáticas. Si utiliza la opción **`-b`** con el comando **`cp`**, sobrescribirá los archivos existentes, pero antes creará una copia de seguridad de los archivos sobrescritos.

```
alumno@so-lab01:~$ cp -b file.txt directorio_objetivo/file.txt
alumno@so-lab01:~$ ls directorio_objetivo
file.txt  file.txt~
```

La copia de seguridad del archivo termina con **`~`**.

También puedes utilizar la opción de actualización **`-u`** cuando se trata de sobrescribir. Con la opción **`-u`**, los archivos de origen sólo se copiarán en la nueva ubicación si el archivo de origen es más reciente que el archivo existente o si no existe en el directorio de destino.

Para resumir:

- `-i` : Confirmar antes de sobrescribir
- `-n` : No sobrescribir
- `-b` : Sobrescribir con la copia de seguridad
- `-u` : Sobrescribir si el archivo de destino es antiguo o no existe

5.2. Copiado recursivo

```
$ cp [opciones] nomentrada ... dirsalida
```

El comportamiento de la copia recursiva depende de la existencia del directorio de salida `dirsalida` y necesita la opción **`-r`** o **`-R`**:

- Si `dirsalida` no existe, la entrada debe consistir en un sólo directorio. Después de crear `dirsalida`, **`cp`** copia todos los ficheros y subdirectorios del directorio de entrada en `dirsalida` para que sea una copia del directorio de entrada.
- Si `dirsalida` ya existe, cada fichero de entrada se copia en el directorio de salida de la misma manera que se hace en la copia no recursiva. Los directorios de entrada se reproducen como subdirectorios del directorio de salida, incluyendo los ficheros y subdirectorios que contengan.

Cómo copiar un directorio en Linux

Puedes utilizar el comando **`cp`** para copiar un directorio en Linux incluyendo todos sus archivos y subdirectorios. En este caso hay que utilizar la opción **`-r`** que significa recursivo.

```
alumno@so-lab01:~$ cp -r directorio_origen directorio_objetivo
```

Si `directorio_objetivo` ya existe, esto copiará todo `directorio_origen` dentro de él, como subdirectorio. Si no existe, `directorio_objetivo` se crea como una copia de `directorio_origen`.

Cómo copiar sólo el contenido de un directorio, no el propio directorio

Continuando con el ejemplo anterior, si ahora hacemos

```
alumno@so-lab01:~$ ls directorio_objetivo
directorio_origen
```

vemos que se ha copiado todo el `directorio_origen` en una nueva ubicación.

Pero si sólo quieres copiar el contenido del directorio de origen en el directorio de destino, debe añadir `/.` al final del directorio de origen. Esto indicará que sólo quieres copiar el contenido del directorio de origen.

Veámoslo con un ejemplo:

```
alumno@so-lab01:~$ ls directorio_origen
archivo_origen_1  archivo_origen_2
```

Ahora copia el contenido del directorio de origen:

```
alumno@so-lab01:~$ cp -r directorio_origen/. directorio_objetivo
```

Si ahora compruebas el contenido del directorio de destino, verás que sólo se ha copiado el contenido del directorio de origen.

```
alumno@so-lab01:~$ ls directorio_objetivo
archivo_origen_1  archivo_origen_2
```

Cómo copiar múltiples directorios

También puedes copiar varios directorios a la vez con el comando cp en Linux.

Sólo tienes que utilizarlo de la misma manera que lo hiciste para un solo directorio.

```
alumno@so-lab01:~$ cp -r directorio_origen_1 directorio_origen_2
directorio_origen_3 directorio_objetivo
```

Siempre es el último argumento del comando el que se toma como directorio de destino.

Si quieres copiar sólo el contenido de varios directorios a la vez, también puedes hacerlo:

```
alumno@so-lab01:~$ cp -r directorio_origen_1/. directorio_origen_2/.
directorio_origen_3/. directorio_objetivo
```

Cómo conservar los atributos al copiar

Cuando se copia un archivo a una nueva ubicación, sus atributos, como los permisos y las marcas de tiempo del archivo, podrán ser diferentes a los del original.

Si deseas conservar los atributos del archivo original, puedes copiar los archivos con la opción **-p**.

Veámoslo con un ejemplo.

```
alumno@so-lab01:~$ ls -l /etc/services
-rw-r--r-- 1 root root 12813 mar 27  2021 /etc/services
```

Si intentas copiar este archivo normalmente, cambiarán su propietario y su fecha:

```
alumno@so-lab01:~$ cp /etc/services .
alumno@so-lab01:~$ ls -l services
-rw-r--r-- 1 alumno alumno 12813 oct  2 21:41 services
```

Pero si uso la opción **p**, el archivo copiado conservará el modo y la marca de tiempo; la propiedad sólo se conserva si quien copia es root.

```
alumno@so-lab01:~$ cp -p /etc/services .
alumno@so-lab01:~$ ls -l services
-rw-r--r-- 1 alumno alumno 12813 mar 27  2021 services
```

Como puedes ver, ha conservado el modo de acceso y la marca de tiempo del archivo fuente con la opción **-p**.

NOTA: La opción **-p** es esencial en la administración de sistemas y en tareas de copia de seguridad. Si se necesita mover o copiar archivos del sistema o de una aplicación, es vital que los permisos, la propiedad y las fechas se mantengan intactos. Un fallo en preservar estos atributos podría causar que una aplicación deje de funcionar o que se produzcan problemas de seguridad.

6. Creación de directorios: la orden mkdir

La orden `mkdir` crea uno o más directorios nuevos. Su sintaxis es:

```
$ mkdir [opciones] dir ...
```

Aquí, cada `dir` es la trayectoria de un directorio que se desea crear. Cada directorio de nueva creación tiene dos entradas: '**.**' para el directorio en sí y '**..**' para su directorio padre. Los permisos del directorio se asignan como `777` y se modifican teniendo en cuenta el

valor **umask**[1] actual, aunque se puede especificar permisos distintos con la opción **-m**. El propietario y el grupo del nuevo directorio serán los del proceso bajo cuyo auspicio se creó el directorio. La creación de un directorio, al igual que la creación de un fichero, requiere permiso de escritura en el directorio padre.

Opciones de la línea de órdenes:

- **-m** o **--mode** (Modo) Esta opción te permite asignar permisos específicos al directorio que estás creando, en lugar de utilizar los permisos por defecto que están definidos por el `umask` del sistema.

Ejemplo:

```
alumno@so-lab01:~$ mkdir -m 700 privado
```

Este comando crea el directorio privado y le asigna los permisos 700 (`rwx------`) en el momento de la creación. Esto significa que solo el propietario puede acceder, leer y escribir en él, y el grupo y los otros usuarios no tienen ningún permiso.

- **-p** o **--parents** (Padres) Esta opción es extremadamente útil. Le indica a `mkdir` que cree los directorios intermedios necesarios en una ruta si no existen. Sin esta opción, si intentas crear un directorio anidado en una ruta que no existe, el comando fallará.

Ejemplo:

```
alumno@so-lab01:~$ mkdir -p proyecto/documentos/informes
```

Si los directorios proyecto y documentos no existen, `mkdir -p` los creará automáticamente antes de crear el directorio final informes. Esto evita tener que crear cada directorio por separado.

No todos los sistemas reconocen estas opciones.

Con la opción **-p** y la expansión de llaves del intérprete ({…}, que no es una expresión regular), es posible crear un árbol completo de directorios con tan sólo una orden. Por ejemplo, si se ejecuta

```
alumno@so-lab01:~$ mkdir -p
Musica/{Pop,Metal/Heavy,BSO,Rock/{Gotico,Progresivo},Clasica/Barroca}
```

Se crea el siguiente árbol de directorios (para ver ese árbol puedes usar la orden **tree**, no está instalada por defecto, para instalarla **sudo apt install tree**):

[1] El uso de umask se estudió en la práctica anterior

```
alumno@so-lab01:~$ tree Musica
Musica
├── BSO
├── Clasica
│   └── Barroca
├── Metal
│   └── Heavy
├── Pop
└── Rock
    ├── Gotico
    └── Progresivo

10 directories, 0 files
```

Comprueba el número de enlaces físicos de cada uno de los directorios creados, ¿cuántos son?, ¿por qué?

Si se ejecuta

```
alumno@so-lab01:~$ mkdir -p Contactos/{Clientes,Proveedores}/{Pedidos,Facturas}
```

Se crea el siguiente árbol de directorios

```
alumno@so-lab01:~$ tree Contactos/
Contactos/
├── Clientes
│   ├── Facturas
│   └── Pedidos
└── Proveedores
    ├── Facturas
    └── Pedidos

7 directories, 0 files
```

Comprueba el número de enlaces físicos de cada uno de los directorios creados, ¿cuántos son?, ¿sabes ya cómo calcular el número de enlaces físicos de un directorio como mínimo?

7. Eliminación de ficheros y directorios: las órdenes rm y rmdir

Las dos órdenes para eliminar ficheros son **`rm`** *(remove)* y **`rmdir`**. **`rm`** puede eliminar ficheros ordinarios y directorios, pero **`rmdir`** sólo puede eliminar directorios. Por otra parte, **`rmdir`** solo puede borrar directorios vacíos. Si el directorio contiene cualquier archivo o subdirectorio, **`rmdir`** mostrará un error y no lo borrará.

La "eliminación" de un fichero, sea un fichero ordinario o un directorio, significa eliminar enlace de un directorio y no eliminar el fichero en sí. Si hay otros enlaces duros con el fichero, se conservan, lo mismo que el fichero. Un fichero sólo se elimina cuando se borra su último enlace duro. Como un directorio es un tipo especial de fichero, está sujeto a las mismas reglas.

7.1. Eliminación de ficheros: la orden rm

La orden **rm** elimina los enlaces con los ficheros, sean ordinarios o directorios. Un fichero se elimina cuando se han borrado todos sus enlaces duros, de manera que la eliminación del único enlace con un fichero borra al propio fichero. La sintaxis de **rm** es:

```
$ rm [opciones] fichero ...
```

donde cada fichero en `fichero ...` puede incluir enlaces a directorios si está presente la opción **-r.** Por omisión, **rm** pide una confirmación si se intenta eliminar un enlace con un fichero que no tiene permiso de escritura. La pregunta de confirmación indica que el fichero está protegido contra escritura. Se puede cambiar este comportamiento con las opciones **-i** y **-f** que se describen más adelante.

rm devuelve un código de salida cero si tiene éxito en la eliminación de todos los enlaces especificados, y un código distinto de cero en caso contrario.

Opciones de la línea de órdenes:

- **-i** Solicita interactivamente la confirmación de cada eliminación. Una respuesta afirmativa (en la configuración española, que comience por «s» o por «y») indica que debe eliminarse el fichero; cualquier otra respuesta indica lo contrario. **rm** solicita esta confirmación incluso si la entrada estándar no es un terminal.
- **-r** Elimina recursivamente los enlaces con directorios, es decir, elimina los enlaces con los directorios y con todos los ficheros y subdirectorios que contengan, directa o indirectamente. De hecho, esta opción elimina toda una rama del árbol de directorios. **rm** pide que se confirme la eliminación de enlaces sin permiso de escritura, pero esta solicitud se omite si la entrada estándar no proviene del terminal, o si se especificó la opción **-f**. Si usted rechaza la eliminación de un enlace, **rm** no eliminará los directorios que contengan ese enlace de forma directa o indirecta.
- **-f** No solicita la confirmación de las eliminaciones, incluso en el caso de enlaces que carecen de permiso de escritura. La opción **-f** tiene precedencia si se especifica junto con la opción **-i**.

NOTA: No se puede eliminar enlaces de un directorio para el cual no se tenga permiso de escritura, sin importar las opciones empleadas.

7.2. Eliminación de directorios: la orden rmdir

La orden rmdir elimina los enlaces con directorios. Su sintaxis es:

```
$ rmdir [opciones] dir ...
```

donde `dir ...` es una lista de directorios.

Por ejemplo, **`rmdir denver`** hace que **`rmdir`** elimine el enlace al subdirectorio `denver` del directorio actual. A diferencia de **`rm -r`**, **`rmdir`** no eliminará un enlace con un directorio si éste no está vacío. **`rmdir`** devuelve un código de salida de cero si tiene éxito en la eliminación de todos los enlaces especificados y un código distinto de cero en caso contrario.

Opciones de la línea de órdenes:

- **`-p o --parents`** (Eliminar Padres) Esta opción es muy potente. Permite eliminar un directorio y, de forma recursiva, todos sus directorios padre si estos quedan vacíos. Es muy útil para limpiar estructuras de directorios que ya no se necesitan.

Ejemplo: Si tienes una ruta como `directorio1/directorio2/directorio3` y el directorio `directorio3` está vacío, puedes usar:

```
alumno@so-lab01:~$ rmdir -p directorio1/directorio2/directorio3
```

Esto eliminará `directorio3`. Si `directorio2` queda vacío, también lo eliminará. Y si `directorio1` también queda vacío, lo borrará, subiendo por la jerarquía hasta encontrar un directorio con contenido.

- **`-v o --verbose`** (modo detallado) Esta opción es muy útil en la práctica. Muestra un mensaje para cada directorio que es eliminado, lo que te permite ver exactamente qué directorios están siendo procesados. Esta opción se usa a menudo en combinación con `-p` para tener un registro de las acciones que se están llevando a cabo.
- **`--ignore-fail-on-non-empty`** (Ignorar fallos en directorios no vacíos) Esta opción evita que rmdir muestre un error y termine su ejecución si encuentra un directorio que no está vacío. Es útil en scripts donde quieres que el comando continúe su ejecución incluso si no puede eliminar un directorio.

8. Localización de ficheros: la orden find

El programa **`find`** busca en las partes especificadas del sistema de ficheros de UNIX los ficheros que cumplen cierto criterio. Este programa tiene multitud de opciones que permiten aplicar criterios complejos como: aquellos ficheros que ocupen más de 10 bloques y su propietario sea distinto del que ejecuta el programa **`find`**. En esta sección sólo se comenta cómo localizar ficheros que coincidan con un patrón de búsqueda, si quiere ampliar sus conocimientos sobre **`find`** consulte un manual o **`man`**.

Para buscar los ficheros que coinciden con un patrón utilizaremos la sintaxis:

```
$ find listrayec -name patrón
```

listrayec es una lista de trayectorias de ficheros y directorios (por lo general, sólo de directorios), indica los ficheros y directorios en los que se debe realizar la búsqueda, si es un directorio la búsqueda será recursiva.

patrón indica el patrón de búsqueda que deben cumplir los ficheros para que find escriba su trayectoria en la salida estándar (absoluta si la trayectoria de partida lo es y relativa en caso contrario). En patrón puede especificar comodines, tal como se explicaron en los temas anteriores.

Por ejemplo, la orden **find /usr -name "v*.h"** envía a la salida estándar una lista de todos los ficheros que están en `/usr` o sus subdirectorios, y que además comiencen con `'v'` y terminen con `'.h'`. La lista podría verse como sigue:

```
alumno@so-lab01:~$ find /usr -name "v*.h"
/usr/lib/x86_64-linux-gnu/perl/5.38.2/CORE/vutil.h
/usr/lib/gcc/x86_64-linux-gnu/13/include/varargs.h
/usr/lib/gcc/x86_64-linux-gnu/13/include/vpclmulqdqintrin.h
/usr/lib/gcc/x86_64-linux-gnu/13/include/vaesintrin.h
/usr/include/rdma/vmw_pvrdma-abi.h
/usr/include/values.h
…
```

Siempre que se **utilizan comodines en el patrón, éste se debe entrecomillar**, ya que los metacaracteres que van entre una expresión entrecomillada no los interpreta el shell. Con esto se logra que **sea find la que interprete los comodines**.

También se puede buscar por el último acceso. Se utiliza la opción **-atime** seguido por un número de días o por el número y un signo + o - delante de él.

-atime 7 busca los archivos a los que se accedió hace 7 días.

-atime -2 busca los archivos a los que se accedió hace menos de 2 días.

-atime +5 busca los archivos a los que se accedió hace más de 5 días.

Otro criterio es por el tipo, se usa la opción **-type** seguida del carácter que indica el tipo de archivo, **f** fichero ordinario, **d** directorio.

```
alumno@so-lab01:~$ find . -type f
```

Se pueden buscar también por su tamaño en bloques de 512 bytes con **-size** seguido de un número y un signo (+ o -).

En el siguiente ejemplo se muestra la búsqueda de los archivos de tamaño igual a 10 bloques o cuyo último acceso se haya efectuado hace más de dos días:

```
alumno@so-lab01:~$ find . -size 10 -o -atime +2
```

Aquí tienes varios ejemplos útiles del comando **`find`** que un administrador de sistemas podría utilizar en su trabajo diario.

Buscar Ficheros por Atributos

Buscar ficheros modificados en las últimas 24 horas:

```
alumno@so-lab01:~$ find /home/alumno -mtime -1  # Ficheros modificados en el último día
```

Buscar ficheros de un tamaño específico:

```
alumno@so-lab01:~$ find /var/log -size +10M  # Ficheros de /var/log de más de 10 megabytes
```

Buscar ficheros con un determinado nombre, sin distinguir mayúsculas/minúsculas:

```
alumno@so-lab01:~$ find /etc -iname "hosts"  # Encuentra hosts, HOSTS o Hosts
```

Buscar Ficheros por Permisos o Propiedad

Buscar ficheros con permisos setuid:

```
alumno@so-lab01:~$ find / -type f -perm -4000 2>/dev/null  # Programas con el bit set-uid activo
```

Buscar todos los ficheros de un usuario concreto:

```
alumno@so-lab01:~$ find /home -user alumno  # Todos los ficheros propiedad del usuario alumno
```

Buscar ficheros con permisos de escritura para el grupo u otros:

```
alumno@so-lab01:~$ find /var/www -perm /g+w,o+w  # Ficheros de /var/www con escritura para el grupo o para otros
```

9. Ejemplos

En este apartado se muestran una serie de ejemplos que aclaran las diferencias entre enlace duro y simbólico, y el uso de algunas órdenes. En ellos se emplea el programa **`cambia`** descrito en el tema anterior que puedes descargar desde aquí:

- `cambia.cpp`

A continuación se muestran los ejemplos:

```
alumno@so-lab01:~$ cd  # Nos desplazamos a nuestro directorio base
alumno@so-lab01:~$ g++ -std=c++17 -Wall -Wextra cambia.cpp -o cambia  # Si no se
hizo en P03
alumno@so-lab01:~$ echo abc > f  # Creamos el fichero f con el contenido abc
alumno@so-lab01:~$ cat f  # Efectivamente, f contiene abc
abc
alumno@so-lab01:~$ ls -li f  # Consultamos información sobre el fichero asociado
al enlace f
33753615 -rw-rw-r-- 1 alumno alumno 4 oct  2 21:42 f
```

La opción **`i`** de **`ls`** hace que el listado muestre el nodo i (en este caso 33753615) de los enlaces listados. Resulta interesante observar la columna que viene tras los permisos, ésta indica el número de enlaces duros al fichero enlazado. En este caso acaba de ser creado, y sólo posee 1 (~alumno/f).

```
alumno@so-lab01:~$ ln f ed  # Creamos un enlace duro de nombre ed al fichero que
enlaza f
alumno@so-lab01:~$ ls -li ed f  # Consultamos información sobre los enlaces ed y
f
33753615 -rw-rw-r-- 2 alumno alumno 4 oct  2 21:42 ed
33753615 -rw-rw-r-- 2 alumno alumno 4 oct  2 21:42 f
```

El nodo i, los permisos, el número de enlaces duros, el propietario y su grupo, el tamaño y la fecha de modificación son los mismos, pues ambos enlaces hacen referencia al mismo fichero. Note que el contador de enlaces duros es ahora dos.

```
alumno@so-lab01:~$ ln -s ~/f es  # Creamos un enlace simbólico de nombre es al
fichero ~/f
alumno@so-lab01:~$ ls -li es f  # Consultamos información sobre los enlaces es y
f
33753616 lrwxrwxrwx 1 alumno alumno 14 oct  2 21:43 es -> /home/alumno/f
33753615 -rw-rw-r-- 2 alumno alumno  4 oct  2 21:42 f
```

Analizando la información proporcionada por **`ls`** podemos deducir que es está enlazado con otro fichero, cuyo nodo i es el 33753616. Además, este fichero se utiliza para almacenar una trayectoria (el primer carácter `l` en el listado de permisos nos dice que es un enlace simbólico) y tiene un único enlace duro (~alumno/es). Su fecha de modificación es posterior a la del fichero enlazado por `f`. Observando la última columna podemos saber su contenido: `/home/alumno/f (su tamaño es 14,` el número de caracteres de la trayectoria que almacena).

```
alumno@so-lab01:~$ ./cambia f a A  # Cambiamos el fichero a través de la entrada
f
✅ Reemplazo completado con éxito en el fichero 'f'.
alumno@so-lab01:~$ ./cambia ed b B  # Cambiamos el fichero a través de la entrada
ed
✅ Reemplazo completado con éxito en el fichero 'ed'.
alumno@so-lab01:~$ ./cambia es c C  # Cambiamos el fichero a través de la entrada
es
✅ Reemplazo completado con éxito en el fichero 'es'.
alumno@so-lab01:~$ cat f ed es  # Observamos el contenido a través de las tres
entradas
ABC
ABC
ABC
```

Como puede deducirse de esta secuencia de órdenes a través de los tres enlaces se accede al mismo fichero.

```
alumno@so-lab01:~$ rm f  # Borramos el primer enlace al fichero
alumno@so-lab01:~$ ls -li ed  # Ahora sólo existe un enlace duro al fichero
33753615 -rw-rw-r-- 1 alumno alumno 4 oct  2 21:43 ed
alumno@so-lab01:~$ ls -li es  # es apunta a /home/alumno/f, que ya no existe:
enlace roto
33753616 lrwxrwxrwx 1 alumno alumno 14 oct  2 21:43 es -> /home/alumno/f
alumno@so-lab01:~$ cat ed es  # Intentamos acceder a través de ed y es
ABC
cat: es: No existe el archivo o el directorio
```

Sólo se puede a través de `ed`, pues `es` intenta acceder a través de la trayectoria `~alumno/f`, y ésta ya no existe.

```
alumno@so-lab01:~$ echo El país > f  # Creamos un fichero nuevo, enlazado con el
nombre f
alumno@so-lab01:~$ cat f ed es
El país
ABC
El país

alumno@so-lab01:~$ cp f copia  # Copiamos el fichero enlazado a f en otro,
enlazado por copia
alumno@so-lab01:~$ ls -li copia f  # Consultamos información sobre los enlaces
copia y f
33753618 -rw-rw-r-- 1 alumno alumno 9 oct  2 21:44 copia
33753617 -rw-rw-r-- 1 alumno alumno 9 oct  2 21:43 f
```

Observe que los ficheros enlazados a `copia` y `f` tienen distinto nodo i, y distinta fecha de modificación.

```
alumno@so-lab01:~$ ./cambia f l L
✅ Reemplazo completado con éxito en el fichero 'f'.
alumno@so-lab01:~$ ./cambia copia p P
✅ Reemplazo completado con éxito en el fichero 'copia'.
alumno@so-lab01:~$ cat copia f
El País
EL país
```

De la última secuencia de órdenes se deduce que cuando se copia un fichero se crea un nuevo fichero, al que se le asocia un nodo i nuevo. Se crea una copia de los datos del fichero fuente en el destino, y las modificaciones en uno de los ficheros no afectan al otro.

10. Ejercicios propuestos

1) Por cada uno de los enunciados siguientes escriba una línea de entrada para el intérprete de órdenes que satisfaga la demanda del enunciado. (Nota: los enunciados deben satisfacerse mediante una única línea de entrada al shell. En general, para satisfacer el enunciado i es preciso que antes haya satisfecho los enunciados que le preceden).

1. Cambie el directorio de trabajo a su directorio base.
2. Muestre en la pantalla el contenido del fichero `/etc/passwd` empleando una trayectoria relativa para hacer referencia al fichero.
3. Cree un directorio de nombre `temp` que cuelgue directamente de su directorio base.
4. Muestre en la pantalla el contenido del directorio creado empleando una trayectoria absoluta para hacer referencia al directorio.
5. Crea en tu directorio base el directorio `dir`
6. Crea con solo una orden la estructura de directorios de la imagen que aparece en 1.3. Directorios y enlaces en el directorio `dir`
7. Crea con solo una orden todos los ficheros que aparecen en esa estructura.
8. Muestre en la pantalla el contenido del fichero `f`. Éste cuelga del directorio `d`, que a su vez cuelga del directorio `dir` que acaba de crear en la pregunta previa.
9. Muestre en la pantalla las trayectorias de los ficheros de nombre que comienzan por `f` que existen en el árbol de directorios que comienza en su directorio base.
10. Muestre en la pantalla el número de ficheros de nombre que comienzan por `f` que existen en el árbol de directorios que comienza en su directorio base. Sugerencia: interconecte la salida de la orden que utilizó en 9) con `wc`.
11. Cree un enlace simbólico en su directorio base al directorio `dir` .
12. Borre el directorio `dir` de su directorio base (con todo su contenido).
13. Cree el directorio `~/d2/d3/d4` con permisos `rwx------`.
14. Copie el fichero `/etc/passwd` a su directorio base.
15. Desplace (mueva) el fichero de su directorio base copiado en 14) al directorio creado en 13).
16. Muestre en la pantalla el contenido del directorio `dir` utilizando el enlace que creó en su directorio en 11).
17. Cree un enlace duro en su directorio base con el fichero que desplazó en 15).
18. Muestre el contenido del fichero asociado al enlace duro que creó en 17).
19. Ejecute el programa `creacap` (lo puedes descargar desde “preguntas frecuentes” de la carpeta “Materiales de prácticas” o de Github).
20. Desplace (o mueva) todos los ficheros (realmente son enlaces) de su directorio base cuyo nombre comience por `cap` al directorio temp (que creó en 3).
21. Muestre en la pantalla el contenido de los ficheros del directorio `temp` cuyo nombre tiene exactamente cuatro caracteres.
22. Muestre en la pantalla el contenido de los ficheros del directorio `temp` cuyo nombre empiece por `cap` seguido de un número entre el 1 y el 7, y que terminan en la cadena .2.

2) Tras satisfacer el enunciado 7) de la pregunta anterior, ¿cuántos enlaces duros existen al directorio dir de su directorio base? ¿Por qué existen ese número de enlaces duros?

3) Compruebe empíricamente si es posible realizar enlaces duros y simbólicos a trayectorias que no existen. ¿Por qué cree que es posible o imposible?

4) ¿Qué ocurre con los permisos de un fichero o directorio cuando se copia?, ¿se preservan los permisos o se aplica la máscara?, ¿de qué forma?, ¿cómo se puede hacer una copia con cp preservando los permisos?

1. Ponga una máscara que quite los permisos de ejecución a los otros.
2. Copia un fichero que tenga los permisos de ejecución activos para todos.
3. Comprueba los permisos del fichero.

Preguntas frecuentes

inodo o nodo i ¿Qué es un inodo o nodo i?

En informática, un inodo, nodo-i, nodo índice o i-node en inglés es una estructura de datos propia de los sistemas de archivos tradicionalmente empleados en los sistemas operativos tipo UNIX como es el caso de Linux. Un inodo contiene las características (permisos, fechas, ubicación, pero NO el nombre) de un archivo regular, directorio, o cualquier otro objeto que pueda contener el sistema de ficheros.

El término "inodo" refiere generalmente a inodos en discos (dispositivos en modo bloque) que almacenan archivos regulares, directorios, y enlaces simbólicos. El concepto es particularmente importante para la recuperación de los sistemas de archivos dañados.

Cada inodo queda identificado por un número entero, único dentro del sistema de ficheros, y los directorios recogen una lista de parejas formadas por un número de inodo y nombre identificativo que permite acceder al archivo en cuestión: cada archivo tiene un único inodo, pero puede tener más de un nombre en distintos o incluso en el mismo directorio para facilitar su localización.

Estructura del inodo

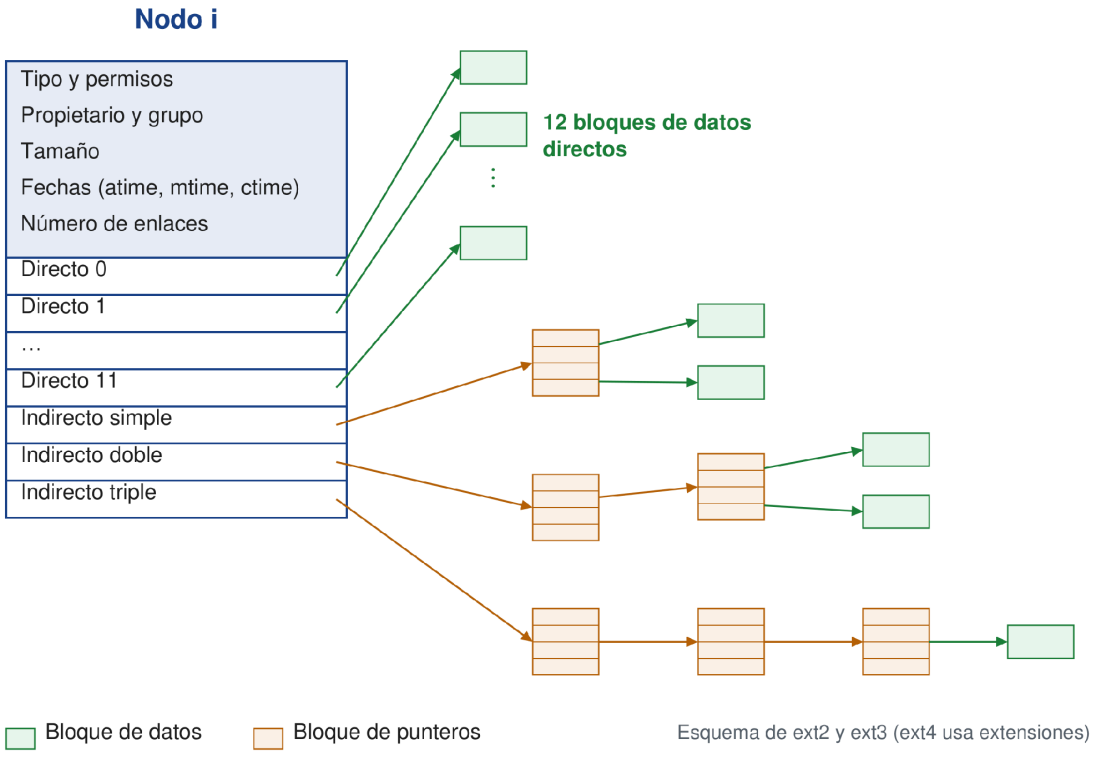
El estándar POSIX establece un modelo de sistema de archivos que se ajusta al empleado en los UNIX tradicionales. Un archivo ordinario tendrá las propiedades siguientes:

- El identificador del dispositivo que alberga al sistema de archivos.
- El número de inodo que identifica al archivo dentro del sistema de archivos
- La longitud del archivo en bytes.
- El identificador de usuario del creador o un propietario del archivo con derechos diferenciados.
- El identificador de grupo de un grupo de usuarios con derechos diferenciados.
- El modo de acceso: capacidad de leer, escribir, y ejecutar el archivo por parte del propietario, del grupo y de otros usuarios.

- Las marcas de tiempo con las fechas de última modificación (mtime), acceso (atime) y de alteración del propio inodo (ctime).
- El número de enlaces, esto es, el número de nombres (entradas de directorio) asociados con este inodo.

El número de enlaces se emplea por el sistema operativo para eliminar el archivo del sistema de ficheros, tanto el inodo como el contenido, cuando se han borrado todos los enlaces y el contador queda a cero.

Ejemplo:



Estructura de un nodo i en ext2 y ext3: punteros a bloques de datos directos e indirectos

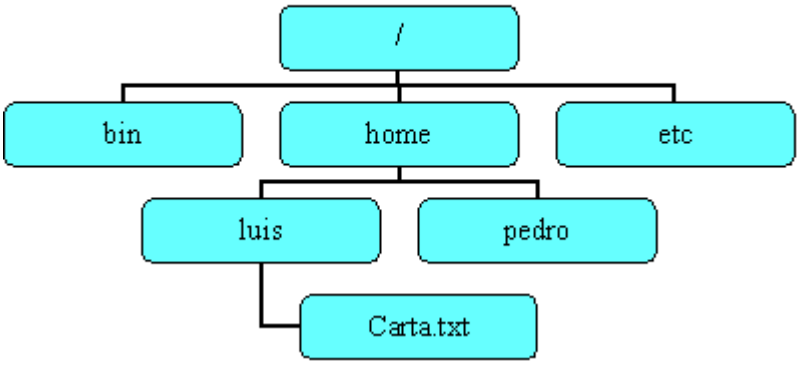
El trabajar sobre un sistema de ficheros basado en inodos resulta al principio chocante para usuarios que no están habituados a él:

- Si múltiples nombres están enlazados, o sea, asociados a un mismo inodo (lo que se denomina enlaces duros o simplemente enlaces) entonces todos los nombres son equivalentes entre sí. El que fue creado en primer lugar no tiene ningún estatus especial, al contrario de lo que ocurre con los enlaces simbólicos o con los denominados accesos directos donde todos dependen del nombre original.
- Tradicionalmente, era posible hacer enlaces (enlaces duros) a directorios. Esto hacía que la estructura de directorios fuera un grafo dirigido en vez de un árbol. Se podía dar la paradoja de que un directorio fuera su propio padre. Los sistemas modernos prohíben generalmente este estado confuso.

Camino o trayectoria ¿Qué es un camino, trayectoria o Path?

En cualquier sistema operativo moderno la **estructura de archivos es jerárquica** y depende de los directorios. En general la estructura del sistema de archivos se asemeja a una estructura de árbol, estando compuesto cada nodo por un directorio o carpeta, que contiene otros directorios o archivos. En Windows cada unidad de disco se identifica como una carpeta básica que sirve de raíz a otras, y cuyo nombre es especial a:, c:, d: etc. En los sistemas Unix, y por lo tanto en Linux, existe una única raíz llamada / de la que cuelgan todos los ficheros y directorios, y que es independiente de qué dispositivos estén conectados al ordenador.

El camino o path de un fichero o directorio es la secuencia de directorios que se ha de recorrer para acceder a un determinado fichero separados por `/`. Supongamos la estructura de archivos de la Figura.



Existen dos formas del path o camino:

- el camino absoluto que muestra toda la ruta a un fichero desde la raíz, siempre comienza con `/`, `/home/luis/Carta.txt`.
- el path relativo a un determinado directorio, por ejemplo si nos encontramos en el directorio `/home`, el path relativo al fichero `Carta.txt` es `luis/Carta.txt`.

Para complicar aún más las cosas, todos los directorios contienen dos directorios especiales o enlaces:

- El directorio actual, representado por el punto `.`
- El directorio padre representado por dos puntos `..`

Estando en el directorio `/home/pedro` se puede acceder a `Carta.txt` con `/home/luis/Carta.txt` (path absoluto) o bien `../luis/Carta.txt` (path relativo). Estando situados en el directorio `luis` como `./Carta.txt` o simplemente `Carta.txt`.